CI/CD

1. What is CI/CD?

CI/CD stands for **Continuous Integration** and **Continuous Delivery/Continuous Deployment**. It is a DevOps practice that automates the **software build, test, and deployment process**.

Goal:
Deliver software **faster, reliably, and automatically**.

Full Form

Term	Full Form	Meaning
CI	Continuous Integration	Developers frequently merge code to a shared repository
CD	Continuous Delivery	Code automatically prepared for release
CD	Continuous Deployment	Code automatically deployed to production

---

2. Continuous Integration (CI)

Definition

Continuous Integration is the process where developers **frequently commit code changes** to a shared repository and **automatically build and test** the application.

CI Process

1. Developer writes code
2. Code pushed to repository (Git)
3. CI tool detects change
4. Code is built
5. Automated tests run
6. Errors reported immediately

Benefits

- Detect bugs early
- Faster development

- Improved code quality
- Automatic testing

Popular CI Tools

- **Jenkins**
- **GitHub Actions**
- **GitLab CI**
- **CircleCI**
- **Travis CI**

---

3. Continuous Delivery (CD)

Definition

Continuous Delivery means the software is **automatically built and tested** and **ready to deploy anytime**.

Deployment to production **requires manual approval**.

Process

```
Code → Build → Test → Staging → Manual Approval → Production
```

Benefits

- Faster releases
- Reliable deployments
- Reduced human errors

---

4. Continuous Deployment

Definition

Continuous Deployment automatically **deploys every change to production** without manual approval.

Process

```
Code → Build → Test → Deploy → Production
```

Example

If a developer pushes code →
System automatically:

- builds application
- runs tests
- deploys to server

---

5. Difference Between CI, Continuous Delivery, Continuous Deployment

Feature	Continuous Integration	Continuous Delivery	Continuous Deployment
Code Integration	Yes	Yes	Yes
Automated Testing	Yes	Yes	Yes
Deployment	No	Manual	Automatic
Production Release	No	With approval	Fully automatic

---

6. CI/CD Pipeline

Definition

A **CI/CD Pipeline** is an automated workflow that moves code from **development → production**.

Pipeline Stages

1. Code Stage
2. Build Stage
3. Test Stage
4. Deploy Stage
5. Monitor Stage

CI/CD Pipeline Flow

```
Developer
   ↓
Git Repository
   ↓
CI Tool (Jenkins)
   ↓
Build
   ↓
Testing
   ↓
Artifact Storage
   ↓
Deployment
   ↓
Production
```

---

7. CI/CD Pipeline Architecture

Typical DevOps pipeline tools:

Stage	Tool
Source Code	Git / GitHub
Build	Maven / Gradle
CI Tool	Jenkins
Artifact Repo	Nexus / Artifactory
Container	Docker
Orchestration	Kubernetes
Cloud	AWS / Azure

---

8. Example CI/CD Pipeline using Jenkins

Step 1 – Developer Push Code

```
git push origin main
```

Step 2 – Jenkins Trigger

Jenkins automatically triggers pipeline.

Step 3 – Build

Example Maven command:

```
mvn clean install
```

Step 4 – Run Tests

```
mvn test
```

Step 5 – Build Docker Image

```
docker build -t myapp .
```

Step 6 – Deploy Application

Example:

```
kubectl apply -f deployment.yaml
```

---

9. CI/CD Workflow Diagram

```
Developer
   ↓
GitHub Repository
   ↓
Jenkins Pipeline
   ↓
Build Application
   ↓
Run Tests
   ↓
Create Artifact
   ↓
Deploy to Server
   ↓
Production
```

---

10. Benefits of CI/CD

- Faster software delivery
- Automated testing
- Reduced manual work
- Faster bug detection
- Continuous feedback
- Reliable releases

---

11. Challenges in CI/CD

- Pipeline configuration complexity
- Tool integration issues
- Infrastructure cost
- Security risks

---

12. Best Practices of CI/CD

1. Maintain single repository
2. Automate build process
3. Use automated testing
4. Keep builds fast
5. Monitor deployments
6. Secure secrets and credentials

---

13. CI/CD Tools in DevOps

Source Code Management

- Git
- GitHub
- GitLab
- Bitbucket

CI Tools

- Jenkins
- GitHub Actions
- CircleCI
- Travis CI

Build Tools

- Maven
- Gradle
- Ant

Containerization

- Docker

Container Orchestration

- Kubernetes

Cloud Platforms

- AWS
- Azure
- Google Cloud

---

14. Example Jenkins CI/CD Pipeline Script

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                sh 'mvn clean install'
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('Deploy') {
            steps {
                sh 'deploy.sh'
            }
        }
    }
}
```

---

15. CI/CD in DevOps Lifecycle

DevOps Lifecycle:

```
Plan
 ↓
Develop
 ↓
Build
```

```
 ↓
Test
 ↓
Release
 ↓
Deploy
 ↓
Operate
 ↓
Monitor
```

CI/CD mainly covers:

- Build
- Test
- Release
- Deploy

---

16. Real Example (Industry Workflow)

Developer writes code →
Push to GitHub →
Jenkins pipeline triggered →
Build using Maven →
Docker image created →
Push image to Docker Hub →
Deploy to Kubernetes →
Application runs in production.

Jenkins

1. What is Jenkins?

Jenkins is an open-source automation server used in DevOps to automate building, testing, and deploying software.

It is mainly used to implement CI/CD pipelines.

Developed by: Kohsuke Kawaguchi
Language: Java

Definition:
Jenkins is a tool that **automates software development processes such as build, test, and deployment.**

---

2. Why Jenkins is Used

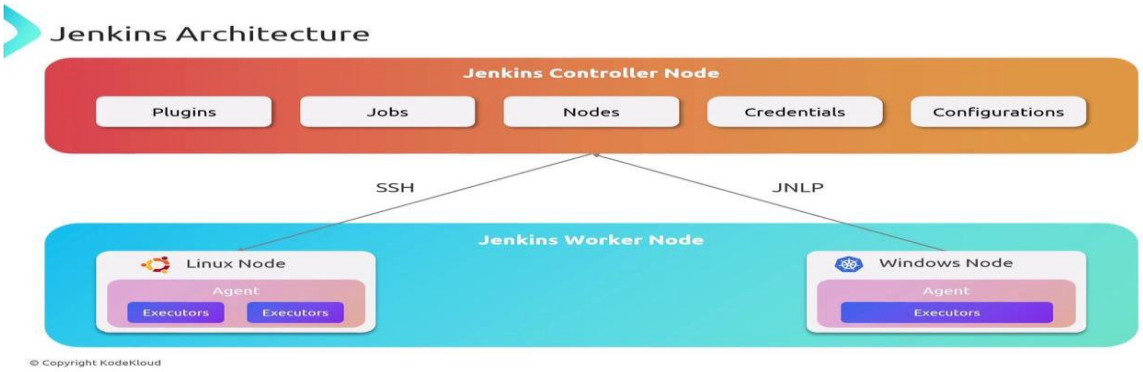
Jenkins helps automate the **software development lifecycle (SDLC)**.

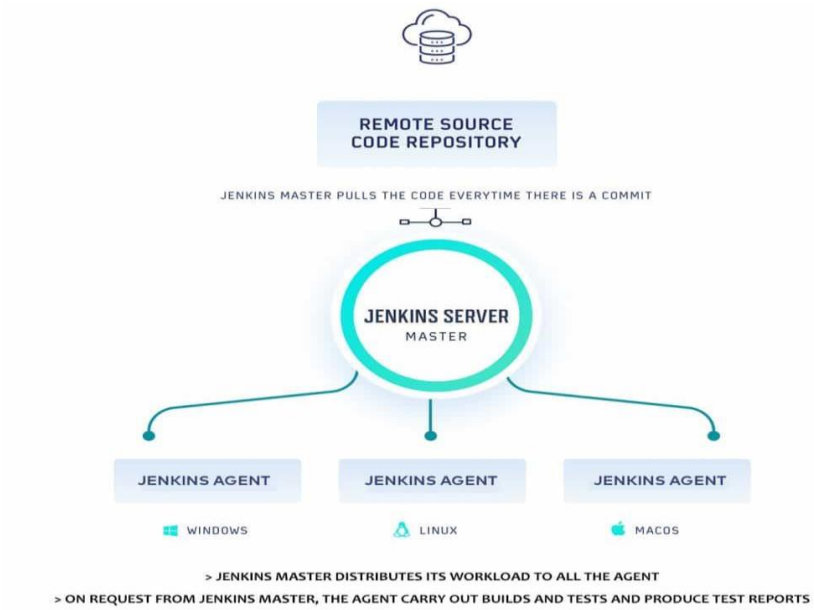
Main Uses

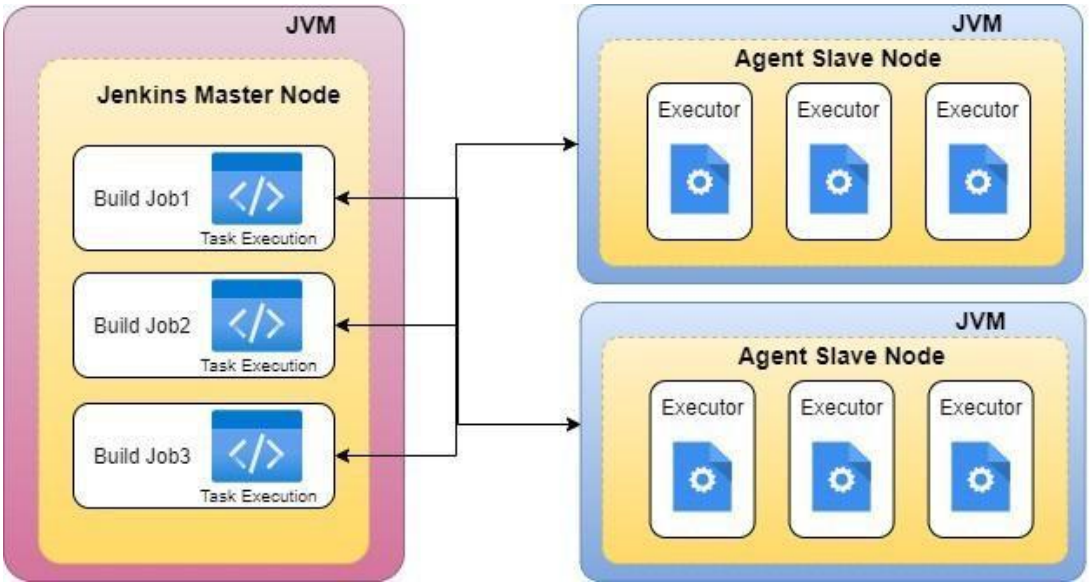
- Automates **build process**
- Runs **automated tests**
- Deploys applications
- Integrates code changes
- Supports **CI/CD pipelines**

---

3. Jenkins Architecture







6

Jenkins works using a **Controller–Agent architecture**.

Components

1. Controller (Master)

- Main Jenkins server
- Manages jobs
- Schedules builds

2. Agent (Slave)

- Executes build jobs
- Connects to controller

3. Jenkins Job
A task or automation process executed by Jenkins.

---

4. Features of Jenkins

- Open source
- Easy integration with DevOps tools
- Large **plugin ecosystem**
- Distributed builds
- Pipeline support
- Automation server
- Cross-platform support

---

5. Jenkins Dashboard

Main sections in Jenkins UI:

- **New Item** – create job
- **Build History** – previous builds
- **Manage Jenkins** – configuration
- **Credentials** – manage secrets
- **Nodes** – agents

---

6. Jenkins Installation (Linux Example)

Since you were recently working with **Amazon Linux**, these are typical steps.

Step 1 – Install Java

```
sudo yum install java-17-amazon-corretto -y
```

Step 2 – Add Jenkins Repository

```
sudo wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/redhat-stable/jenkins.repo
```

Step 3 – Import Jenkins Key

```
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
```

Step 4 – Install Jenkins

```
sudo yum install jenkins -y
```

Step 5 – Start Jenkins

```
sudo systemctl start jenkins
```

Step 6 – Enable Jenkins

```
sudo systemctl enable jenkins
```

---

7. Jenkins Default Port

Jenkins runs on:

```
http://<server-ip>:8080
```

Example:

```
http://54.xx.xx.xx:8080
```

---

8. Jenkins Initial Setup

Get admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Steps:

1. Open Jenkins URL
2. Enter admin password
3. Install suggested plugins
4. Create admin user
5. Start Jenkins

---

9. Jenkins Job Types

1. Freestyle Project

Simple Jenkins job.

2. Pipeline

Pipeline defined using code.

3. Multibranch Pipeline

Builds branches automatically.

4. Folder

Organizes Jenkins jobs.

---

10. Jenkins Pipeline

A **pipeline** defines the **CI/CD workflow**.

Pipeline stages include:

1. Build
2. Test
3. Deploy

Example pipeline flow:

```
Code → Build → Test → Deploy
```

---

11. Jenkins Pipeline Script Example

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                echo 'Building application'
            }
        }
```

```
        stage('Test') {
            steps {
                echo 'Running tests'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploying application'
            }
        }
    }
}
```

---

12. Jenkins Plugins

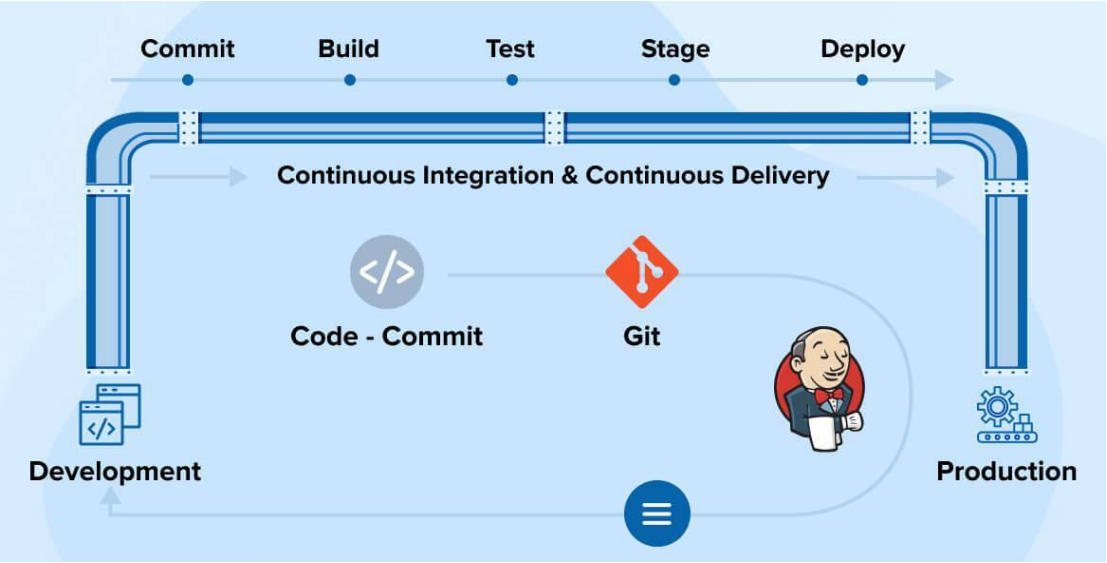
Jenkins has **2000+ plugins**.

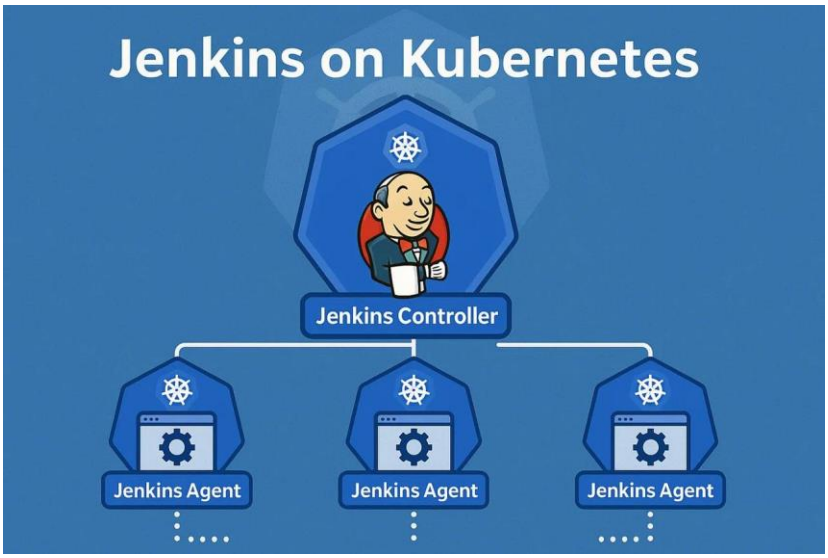
Popular plugins:

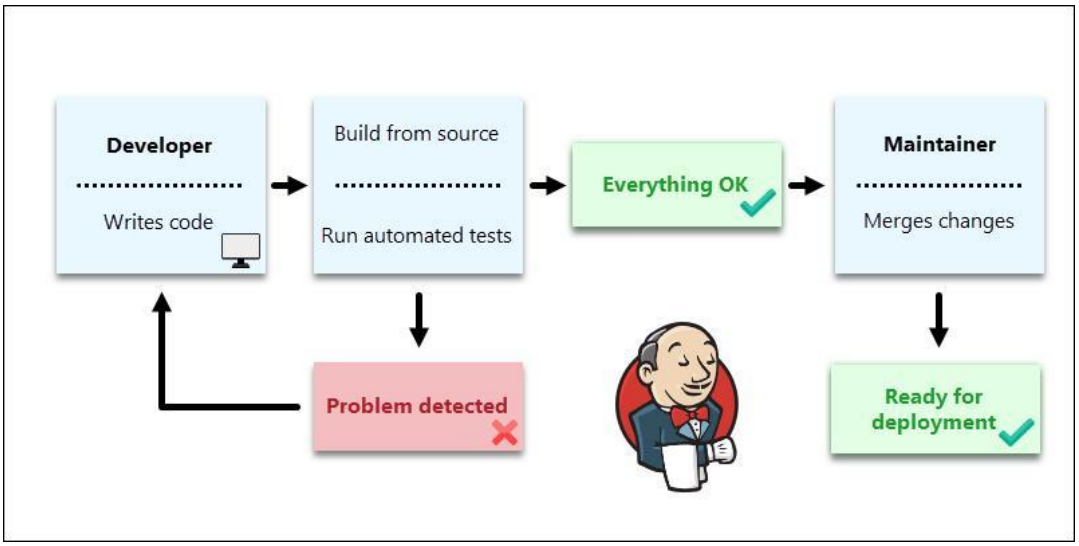
- Git Plugin
- Docker Plugin
- Kubernetes Plugin
- Maven Plugin
- Pipeline Plugin
- Blue Ocean Plugin

---

13. Jenkins CI/CD Workflow







8

Workflow:

1. Developer pushes code to **GitHub**
2. Jenkins detects change
3. Jenkins pulls code
4. Application build starts
5. Tests executed
6. Deployment to server

---

14. Advantages of Jenkins

- Free and open source
- Easy integration with DevOps tools
- Supports CI/CD
- Large community support
- Highly customizable

---

15. Disadvantages of Jenkins

- Requires maintenance
- Complex configuration
- UI not modern
- Plugin dependency issues

---

16. Jenkins vs Other CI/CD Tools

Tool	Type
Jenkins	Open source CI/CD
GitHub Actions	Cloud CI/CD
GitLab CI	Integrated CI/CD
CircleCI	Cloud CI/CD

---

17. Jenkins in DevOps Pipeline

Typical DevOps architecture:

```
GitHub
   ↓
Jenkins
   ↓
Build Tool (Maven)
   ↓
Docker
   ↓
Kubernetes
   ↓
AWS Cloud
```

---

18. Important Jenkins Interview Questions

1. What is Jenkins?
Jenkins is an open-source automation server used for CI/CD.

2. What is Jenkins pipeline?
Automation workflow that defines build, test, and deployment stages.

3. What is Jenkins agent?
A machine that executes Jenkins build jobs.

4. What language is Jenkins written in?
Java.

5. What port Jenkins runs on?
8080.

✅Since you are learning **DevOps tools like Terraform, AWS, and Jenkins**, the **most common real pipeline** is:

```
GitHub → Jenkins → Maven → Docker → Kubernetes → AWS
```